VIETNAM NATIONAL UNIVERSITY - HO CHI MINH CITY
HO CHI MINH CITY UNIVERSITY OF TECHNOLOGY
FACULTY OF COMPUTER SCIENCE AND ENGINEERING

MICROPROCESSOR MICROCONTROLLER (CO3009)

LAB 5

Flow and Error Control in Communication

Class: DT03 – Semester HK251
Submission Date: 28/11/2025

Instructor: Mr. Ton Huynh Long

Student Name	Student ID
Phạm Công Võ	2313946

Ho Chi Minh City, September 2025

Table of Contents

List of Figures

1 Introduction

Flow control and Error control are the two main responsibilities of the data link layer, which is a communication channel for node-to-node delivery of the data. The functions of the flow and error control are explained as follows.

Flow control mainly coordinates with the amount of data that can be sent before receiving an acknowledgment from the receiver and it is one of the major duties of the data link layer. For most of the communications, flow control is a set of procedures that mainly tells the sender how much data the sender can send before it must wait for an acknowledgment from the receiver.

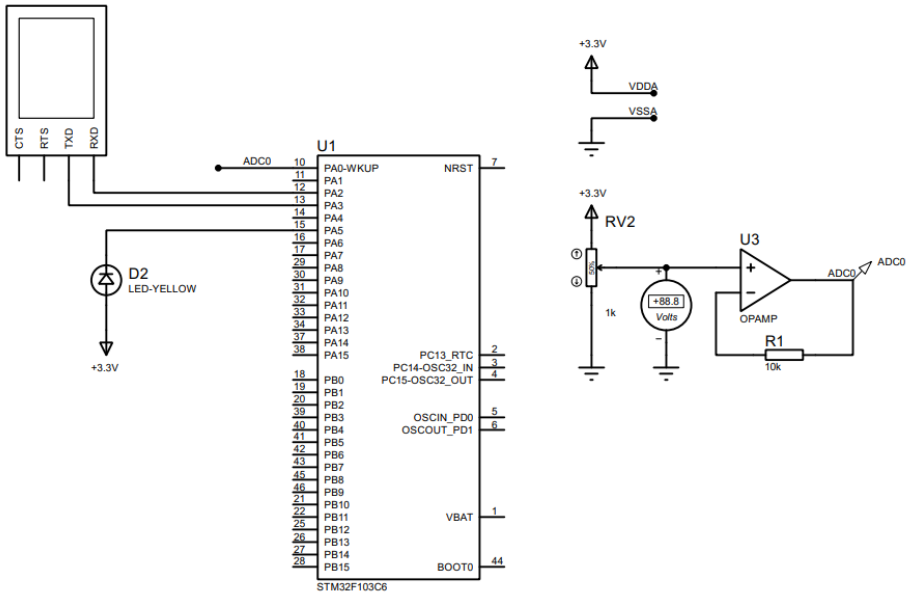
A critical issue, but not really frequently occurred, in the flow control is that the processing rate is slower than the transmission rate. Due to this reason each receiving device has a block of memory that is commonly known as buffer, that is used to store the incoming data until this data will be processed. In case the buffer begins to fill-up then the receiver must be able to tell the sender to halt the transmission until once again the receiver become able to receive.

Meanwhile, error control contains both error detection and error correction. It mainly allows the receiver to inform the sender about any damaged or lost frames during the transmission and then it coordinates with the re-transmission of those frames by the sender.

The term Error control in the communications mainly refers to the methods of error detection and re-transmission. Error control is mainly implemented in a simple way and that is whenever there is an error detected during the exchange, then specified frames are re-transmitted and this process is also referred to as Automatic Repeat request(ARQ).

The target in this lab is to implement a UART communication between the STM32 and a simulated terminal. A data request is sent from the terminal to the STM32. Afterward, computations are performed at the STM32 before a data packet is sent to the terminal. The terminal is supposed to reply an ACK to confirm the communication successfully or not.

2 Proteus simulation platform



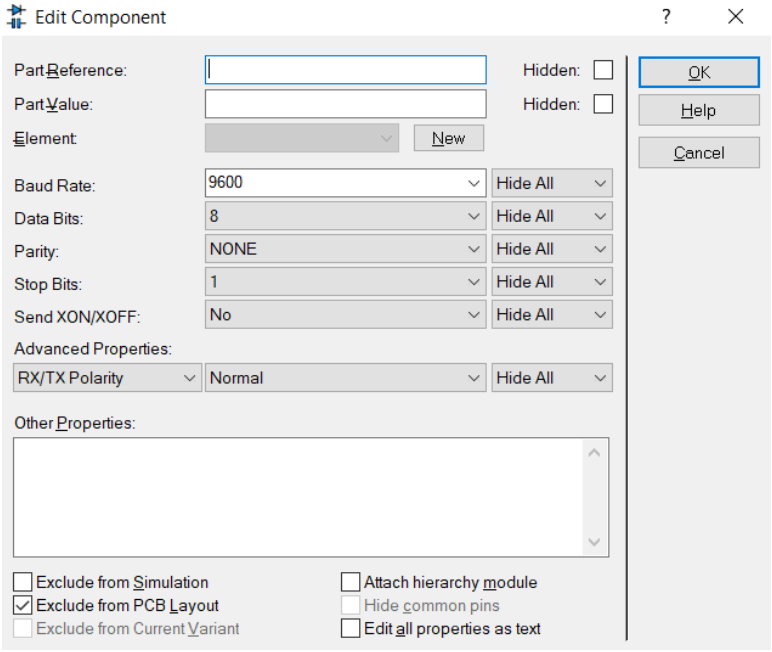
Hình 1: *Simulation circuit on Proteus*

Some new components are listed bellow:

- Terminal: Right click, choose Place, Virtual Instrument, then select VIRTUAL TERMINAL.
- Variable resistor (RV2): Right click, choose Place, From Library, and search for the POT-HG device. The value of this device is set to the default 1k.
- Volt meter (for debug): Right click, choose Place, Virtual Instrument, the select DC VOLTMETER.
- OPAMP (U3): Right click, choose Place, From Library, and search for the OPAMP device.

The opamp is used to design a voltage follower circuit, which is one of the most popular applications for opamp. In this case, it is used to design an adc input signal, which is connected to pin PA0 of the MCU.

Double click on the virtual terminal and set its baudrate to 9600, 8 data bits, no parity and 1 stop bit, as follows:



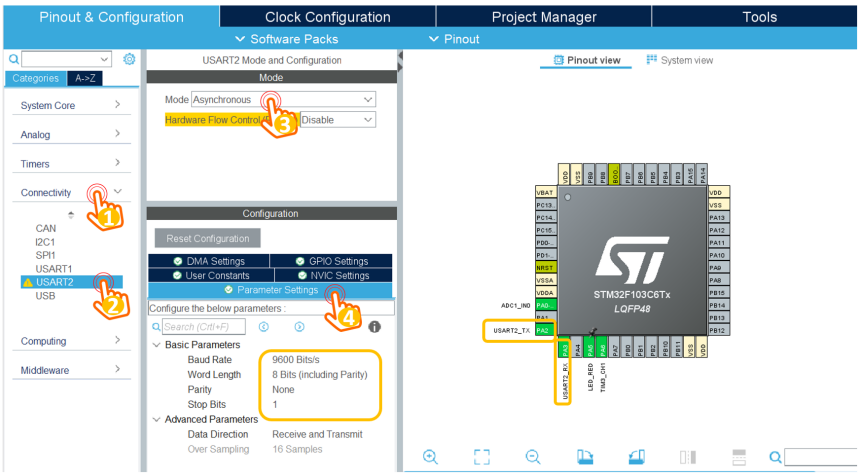
Hình 2: *Terminal configuration*

3 Project configurations

A new project is created with following configurations, concerning the UART for communications and ADC input for sensor reading. The pin PA5 should be an GPIO output, for LED blinky.

3.1 UART Configuration

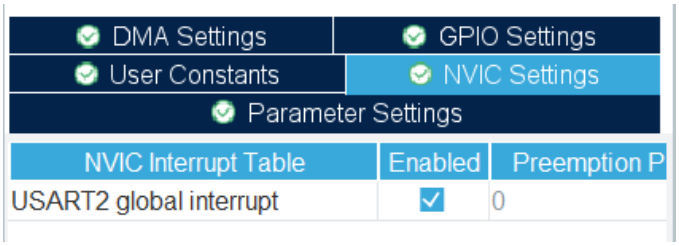
From the ioc file, select **Connectivity**, and then select the **USART2**. The parameter settings for UART channel 2 (USART2) module are depicted as follows:



Hình 3: *UART configuration in STMCube*

The UART channel in this lab is the Asynchronous mode, 9600 bits/s with no Parity and 1 stop bit. After the uart is configured, the pins PA2 (Tx) and PA3(Rx) are enabled.

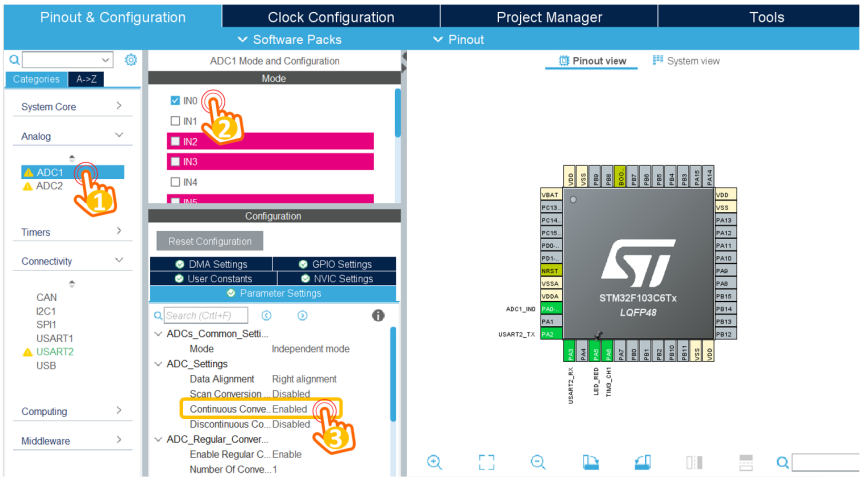
Finally, the NVIC settings are checked to enable the UART interrupt, as follows:



Hình 4: *Enable UART interrupt*

3.2 ADC Input

In order to read a voltage signal from a simulated sensor, this module is required. By selecting on **Analog**, then **ADC1**, following configurations are required:



Hình 5: *Enable UART interrupt*

The ADC pin is configured to PA0 of the STM32, which is shown in the pinout view dialog.

Finally, the PA5 is configured as a GPIO output, connected to a blinky LED.

4 UART loop-back communication

This source is required to add in the main.c file, to verify the UART communication channel: sending back any character received from the terminal, which is well-known as the loop-back communication.

```
/* USER CODE BEGIN 0 */
uint8_t temp = 0;

void HAL_UART_RxCpltCallback(UART_HandleTypeDef *huart){
  if(huart->Instance == USART2){
    HAL_UART_Transmit(&huart2, &temp, 1, 50);
    HAL_UART_Receive_IT(&huart2, &temp, 1);
  }
}
```

```
/* USER CODE END 0 */
```

When a character (or a byte) is received, this interrupt service routine is invoked. After the character is sent to the terminal, the interrupt is activated again. This source code should be placed in a user-defined section.

Finally, in the main function, the proposed source code is presented as follows:

```
int main(void)
{
  HAL_Init();
  SystemClock_Config();

  MX_GPIO_Init();
  MX_USART2_UART_Init();
  MX_ADC1_Init();

  HAL_UART_Receive_IT(&huart2, &temp, 1);

  while (1)
  {
    HAL_GPIO_TogglePin(LED_RED_GPIO_Port, LED_RED_Pin);
    HAL_Delay(500);
  }

}
```

5 Sensor reading

A simple source code to read adc value from PA0 is presented as follows:

```
uint32_t ADC_value = 0;
while (1)
{
  HAL_GPIO_TogglePin(LED_RED_GPIO_Port, LED_RED_Pin);
  ADC_value =  HAL_ADC_GetValue(&hadc1);
HAL_UART_Transmit(&huart2, (void *)str, sprintf(str, "%d\n",
   ADC_value), 1000);
  HAL_Delay(500);
}
```

Every half of second, the ADC value is read and its value is sent to the console. It is worth noticing that the number ADC_value is convert to ascii character by using the sprintf function.

The default ADC in STM32 is 13 bits, meaning that 5V is converted to 4096 decimal value. If the input is 2.5V, ADC_value is 2048.

6 Project description

In this lab, a simple communication protocol is implemented as follows:

- From the console, user types **!RST#** to ask for a sensory data.
- The STM32 response the ADC_value, following a format **!ADC=1234#**, where 1234 presents for the value of ADC_value variable.
- The user ends the communication by sending **!OK#**

The timeout for waiting the **!OK#** at STM32 is 3 seconds. After this period, its packet is sent again. **The value is kept as the previous packet**.

6.1 Command parser

This module is used to received a command from the console. As the reception process is implement by an interrupt, the complexity is considered seriously. The proposed implementation is given as follows.

Firstly, the received character is added into a buffer, and a flag is set to indicate that there is a new data.

```
#define MAX_BUFFER_SIZE  30
uint8_t temp = 0;
uint8_t buffer[MAX_BUFFER_SIZE];
uint8_t index_buffer = 0;
uint8_t buffer_flag = 0;
void HAL_UART_RxCpltCallback(UART_HandleTypeDef *huart){
  if(huart->Instance == USART2){

    //HAL_UART_Transmit(&huart2, &temp, 1, 50);
    buffer[index_buffer++] = temp;
    if(index_buffer == 30) index_buffer = 0;

    buffer_flag = 1;
    HAL_UART_Receive_IT(&huart2, &temp, 1);
  }
}
```

A state machine to extract a command is implemented in the while(1) of the main function, as follows:

```
while (1){
    if(buffer_flag == 1){
        command_parser_fsm();
        buffer_flag = 0;
    }
}
```

The output of the command parser is to set **command_flag** and **command_data**. In this project, there are two commands, **RTS** and **OK**. The program skeleton is proposed as follows:

```
while (1){
    if(buffer_flag == 1){
        command_parser_fsm();
        buffer_flag = 0;
    }
    uart_communiation_fsm();
}
```

6.2 Project implementation

Students are proposed to implement 2 FSM in seperated modules. Students are asked to design the FSM before their implementations in STM32Cube.

The complete LAB5 project source code is available at: GitHub - Flow and Error Control in Communication..

6.2.1 Lab Objectives: Building a Reliable Communication System

The primary goal of this lab is to implement a robust UART system encompassing four core functionalities:

1. **Flow Control (ACK-based)**: Utilize an Acknowledgment (ACK) mechanism (`!OK#`) to coordinate transmission. Ensure the transmitter (STM32) only sends data when the receiver (Terminal) is ready and has acknowledged the previous packet.

2. **Error Control (Timeout ARQ)**: Establish an ARQ mechanism based on a **3-second** Timeout. The system must automatically detect communication failure and perform **Retransmission** of the stored data packet until a valid ACK is received.

3. **Command Parser (FSM-driven)**: Develop a smart command parser using an FSM to recognize commands in the standard format `!<COMMAND_NAME >#`. Effectively filter out unwanted noise characters from the raw UART data stream.

4. **Sensor Reading (ADC Integration)**: Integrate the ADC (Analog-to-Digital Converter) module to read analog voltage (0-5V) from pin PA0. Convert the analog signal into a digital value ($0 \rightarrow 4095$) and serialize it into an ASCII string.

6.2.2 Implementation Requirements and FSM Architecture

The system is designed with a modular architecture featuring two independent FSMs to manage command parsing and protocol handling separately.

*** Finite State Machine (FSM) Design**

(a) **Command Parser FSM (Input Processing)**

- **Function**: Identify and validate commands from the UART Circular Buffer.
- **Core States**:
  - `PARSER_INIT`: Waiting for the start delimiter `!`.
  - `PARSER_WAIT_COMMAND`: Collecting the command string, waiting for the end delimiter `#`.
- **Supported Commands**:
  - `!RST#`: Request to read and transmit ADC data.
  - `!OK#`: Acknowledgment (ACK) command.

(b) **UART Communication FSM (Protocol Management)**

- **Function**: Coordinate the communication flow, implement timeout, and handle ARQ.
- **Core States**:
  - `COMM_WAIT_RST`: Ready to receive a new `!RST#` request.
  - `COMM_WAIT_OK`: Data (`!ADC=xxxx#`) has been sent, the 3-second Timer is running, waiting for `!OK#`.
- **ARQ Feature**: Uses the previously read and stored ADC value for retransmission, avoiding unnecessary re-activation of the ADC module during error recovery.

*** Communication Protocol (3-Way Handshake)**

The communication sequence must strictly adhere to the **Request-Response-ACK** model:

Step	Device	Content	Purpose
1	Terminal → STM32	`!RST#`	Request STM32 to read ADC data.
2	STM32 → Terminal	`!ADC=xxxx#` (e.g., `!ADC=2048#`)	Send current ADC value (0–4095). Start the **3-second Timeout Timer**.
3	Terminal → STM32	`!OK#`	ACK received. Stop Timer. STM32 sends `[SUCCESS]` and returns to `IDLE`.

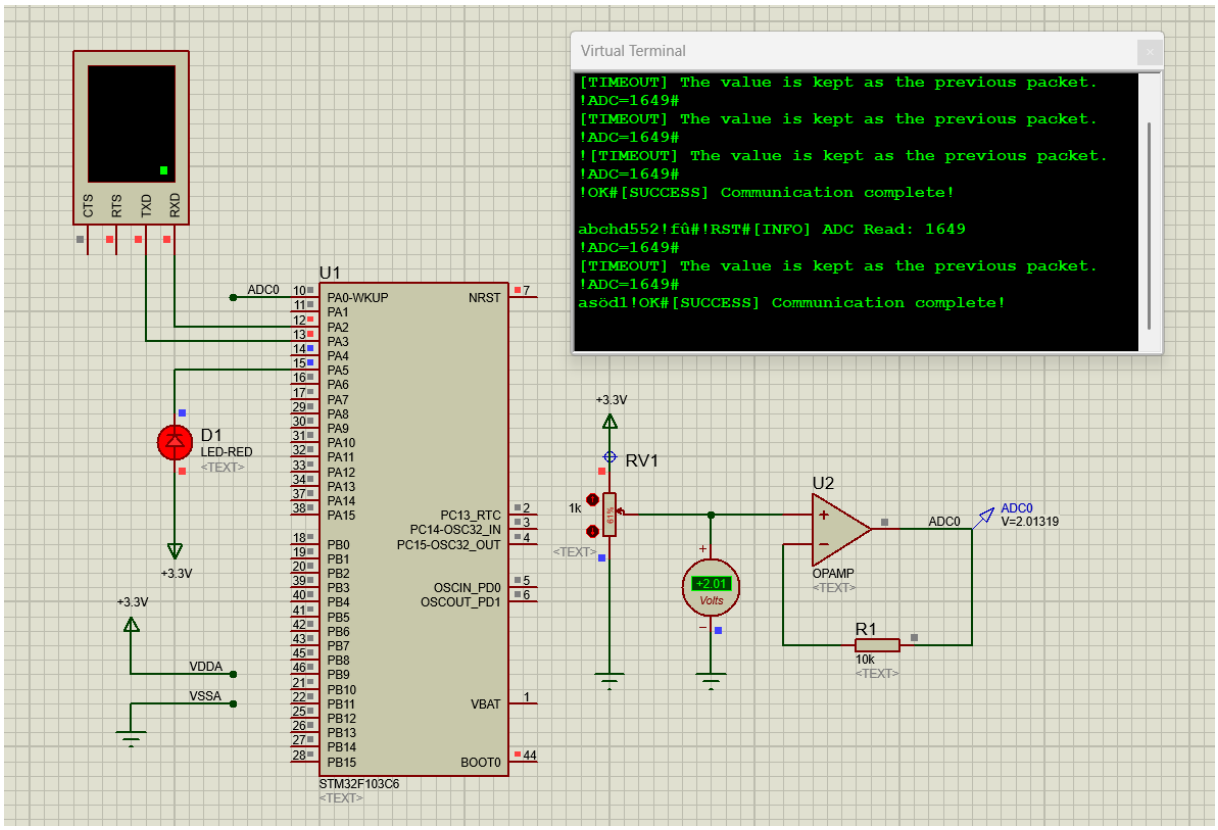
*** Error Handling Mechanism (Timeout-based Retransmission)**

The system implements the ARQ (Automatic Repeat Request) mechanism with the following properties:

- **Timeout**: 3000 milliseconds (3 seconds).
- **Error Detection**: If the `COMM_WAIT_OK` state persists beyond 3 seconds (Timer expiration), a timeout error is triggered.
- **Recovery (Retransmission)**:
  - STM32 sends the error message: `[TIMEOUT] The value is kept as the previous packet.`
  - Automatically retransmits the stored `!ADC=xxxx#` packet.
  - Resets the Timer and continues waiting for `!OK#`.
- **Attempts**: Unlimited retransmission attempts until a valid ACK is received.

6.2.3 Proteus Simulation

The simulation circuit is designed to establish a realistic test environment for the UART Flow Control/ARQ system on the STM32.



Hình 6: *Proteus of System*

Bảng 2: *Technical Analysis of the Proteus Simulation Circuit*

Component	Primary Role in Lab	Specific Technical Function
STM32F103C6	Protocol Controller	The central microcontroller executing the logic of the 2 FSMs (Parser and Communication). Handles UART , ADC, and Timer-based ARQ/Timeout.
Virtual Terminal	HMI & Control Interface	Simulates the PC side. Generates Commands (`!RST#`, `!OK#`) and displays `!ADC=xxxx#` via UART.
Potentiometer (POT)	Analog Sensor Simulation	Provides a variable voltage ($0 \rightarrow V_{CC}$) as analog sensor input for the ADC module.
OPAMP (Voltage Follower)	ADC Input Optimization	Acts as a buffer to prevent loading effect and ensure low output impedance for ADC input (**PA0**). Improves A/D conversion accuracy.
LED (PA5)	Debug Status Indicator	Provides visual feedback for the Communication FSM: lights during `WAIT_FOR_ACK` or on Timeout errors.

6.2.4 TWO STATE MACHINES (2 FSM)

This advanced UART communication system is architected around the Finite State Machine (FSM) model to ensure modularity, robust error handling, and effective data flow management (Flow Control). The system utilizes two (2) independent FSMs operating in parallel to clearly separate input processing responsibilities from protocol management.

a) Command Parser FSM (Input Processor)

The Command Parser FSM is designed to process the raw data stream received via UART. Its main role is noise filtering and command pattern recognition based on the required structure: `!COMMAND_NAME#`.

Bảng 3: *Command Parser FSM States*

State	Description	Primary Action
`PARSER_INIT`	**Initialization.** The FSM waits for the start delimiter `!`.	Ignore all characters that are not `!`.
`PARSER_WAIT_COMMAND`	**Command Collection State.** After receiving `!`, the FSM collects subsequent characters.	Store incoming characters into the temporary command buffer (`temp_cmd`).

* State Transition Logic (Transitions)

The FSM uses a temporary buffer (`temp_cmd`) and an index (`cmd_index`) to construct the command string.

Bảng 4: *Command Parser FSM Transition Table*

From State	To State	Activation Condition	Action
PARSER_INIT	PARSER_WAIT_COMMAND	Character ! is received	Clear buffer (temp_cmd) and set cmd_index = 0. Start command collection.
PARSER_WAIT_COMMAND	PARSER_INIT	Character # is received	**Valid Command:** Compare string: "RST" → CMD_RST, "OK" → CMD_OK. Reset to wait state.
PARSER_WAIT_COMMAND	PARSER_WAIT_COMMAND	Character other than ! or #	Store character in temp_cmd and increment cmd_index.
PARSER_WAIT_COMMAND	PARSER_WAIT_COMMAND	Character ! is received	**Command Restart:** Early format error. Clear buffer and start collecting new command.
PARSER_WAIT_COMMAND	PARSER_INIT	Buffer Overflow	**Buffer/Format Error:** Discard current command and reset to safe state.

Example: Command Parsing Flow

```
Input: "abc!RST#xyz"

a -> PARSER_INIT (ignored)
b -> PARSER_INIT (ignored)
c -> PARSER_INIT (ignored)
! -> PARSER_WAIT_COMMAND (start collecting)
R -> temp_cmd = "R"
S -> temp_cmd = "RS"
T -> temp_cmd = "RST"
# -> Compare "RST" == "RST"
     => command_flag = CMD_RST
x -> PARSER_INIT (ignored)
y -> PARSER_INIT (ignored)
z -> PARSER_INIT (ignored)
```

b) Communication FSM (Protocol Manager)

The Communication FSM serves as the central control logic for the entire protocol. It is responsible for coordinating the 3-Way Handshake procedure and executing the Automatic Repeat Request (ARQ) mechanism based on a strict Timeout period.

The main objective of this FSM is to Control the communication process between the STM32 and the Terminal (PC) according to the defined protocol, ensuring data integrity and reliability.

Bảng 5: *Communication FSM States and Timeout Configuration*

State	Description	Core Technical Role	Timeout
`COMM_IDLE`	**Initial State.** System is ready to begin a new communication cycle.	Waits for a trigger to transition into the command-waiting sequence.	None
`COMM_WAIT_RST`	**Waiting for Data Request.** Awaits `!RST#` from Terminal.	Filters irrelevant commands and ensures correct protocol synchronization.	None
`COMM_WAIT_OK`	**Waiting for ACK.** STM32 has sent ADC data and is waiting for `!OK#`.	Ensures **Flow Control** (no new request allowed) and **Error Control** (timeout tracking).	3 seconds
`COMM_TIMEOUT`	**Error Handling State.** Timeout occurred in `COMM_WAIT_OK`.	Performs error recovery (ARQ) before initiating retransmission.	None

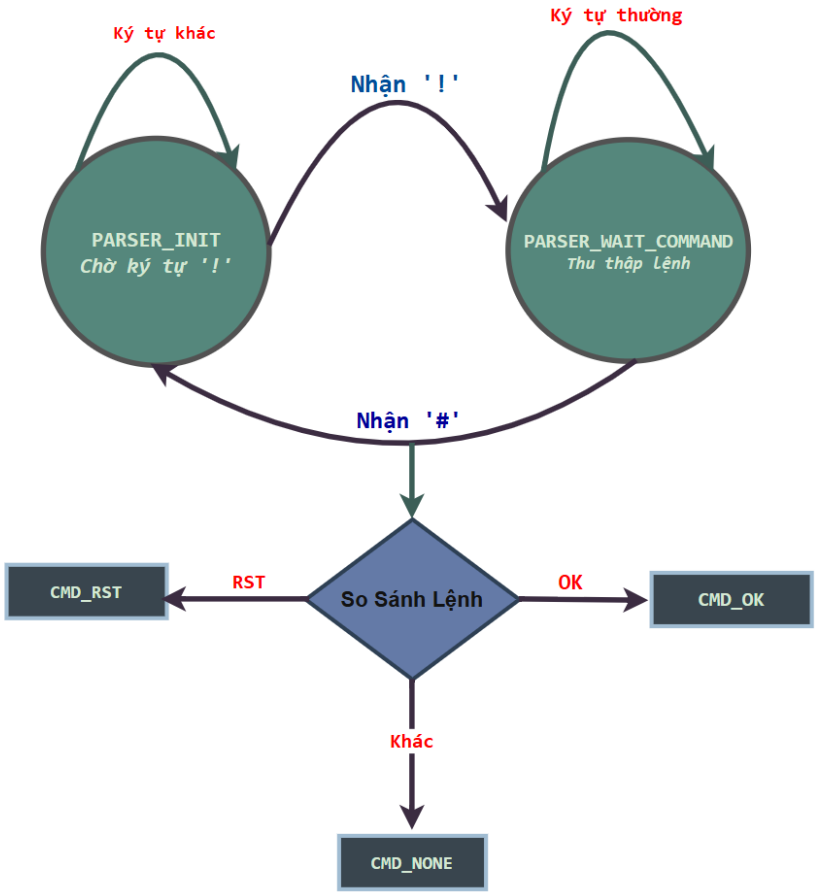
* State Transition Logic (Transitions)

The transitions illustrate how the Communication FSM manages Flow Control (by requiring an ACK) and initiates the ARQ mechanism to ensure reliability.

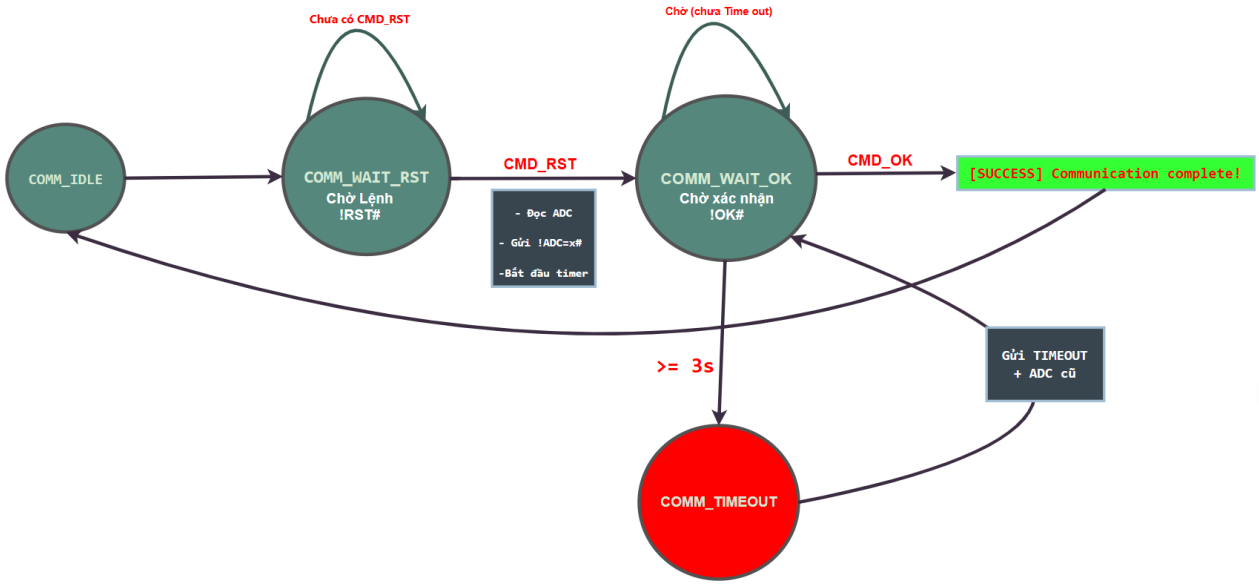
Bảng 6: *Communication FSM Transition Table*

From State	To State	Activation Condition	Action
`COMM_IDLE`	`COMM_WAIT_RST`	Automatic	Initializes the FSM for a new communication cycle.
`COMM_WAIT_RST`	`COMM_WAIT_OK`	`CMD_RST` event received	1) Read ADC. 2) Send `!ADC=xxxx#`. 3) Start 3s timer.
`COMM_WAIT_OK`	`COMM_WAIT_RST`	`CMD_OK` event received (ACK)	1) Stop timer. 2) Send `[SUCCESS]`. . . . 3) Wait for next `!RST#`.
`COMM_WAIT_OK`	`COMM_TIMEOUT`	Timer ≥ 3 s	Switch to ARQ timeout handling.
`COMM_TIMEOUT`	`COMM_WAIT_OK`	Automatic	1) Send `[TIMEOUT]`. . . . 2) Retransmit ADC value. 3) Reset and restart 3s timer.

c) Two FSM Architecture



Hình 7: *Command Parser FSM*



Hình 8: *Communication FSM*

6.2.5 Source Code

1. `command_parser.c`

```c
/* command_parser.c - Command parser using FSM for UART data
 * FUNCTION: Parse commands from UART buffer in format: !COMMAND#
 * SUPPORTED COMMANDS: !RST# and !OK#
 */
#include "command_parser.h"
#include "string.h"

// GLOBAL VARIABLES
static ParserState parser_state = PARSER_INIT;

// Current position in temp buffer
static uint8_t cmd_index = 0;

// Temporary buffer for command being received
static uint8_t temp_cmd[MAX_CMD_LENGTH];

// Last processed position in UART buffer
static uint8_t last_processed_index = 0;

// INITIALIZATION
void command_parser_init(void)
{
    parser_state = PARSER_INIT;     // FSM starts waiting for '!'
    cmd_index = 0;                  // No characters in temp buffer
   yet
    command_flag = CMD_NONE;        // No command received yet
    last_processed_index = 0;       // Start from beginning of buffer

    // Clear all buffers
    memset(command_data, 0, MAX_CMD_LENGTH);
    memset(temp_cmd, 0, MAX_CMD_LENGTH);
}


```

```
/*
1. Iterate through ALL new characters in buffer (not yet processed)
2. For each character, process according to current state
3. Update state and temp buffer */
void command_parser_fsm(void)
{
    // STEP 1: Iterate through ALL new characters in buffer
    while (last_processed_index != index_buffer)
    {
        // Get character at unprocessed position
        uint8_t received_char = buffer[last_processed_index];

        // Move pointer to next position (circular buffer)
        last_processed_index++;
        if (last_processed_index >= MAX_BUFFER_SIZE)
        {
            last_processed_index = 0; // Wrap around to beginning
        }

        // STEP 2: FSM state
        switch (parser_state)
        {
        // STATE 1: PARSER_INIT - Waiting for start character '!'
        case PARSER_INIT:
            if (received_char == '!')
            {
                // Detected command start
                parser_state = PARSER_WAIT_COMMAND;
                cmd_index = 0;
                memset(temp_cmd, 0, MAX_CMD_LENGTH);
            }
            // Other characters - Ignore
            // Example: If receiving "abc!" then ignore "a", "b", "c
   "
            break;

```

```
69          // STATE 2: PARSER_WAIT_COMMAND
70          case PARSER_WAIT_COMMAND:
71              // CASE 1: Received end character '#'
72              if (received_char == '#')
73              {
74                  // Null-terminate string in temp_cmd
75                  temp_cmd[cmd_index] = '\0';
76                  // Compare received command with valid commands
77                  // EXAMPLES:
78                  // Receive "!RST#" - temp_cmd = "RST" - CMD_RST
79                  // Receive "!OK#"  - temp_cmd = "OK"  - CMD_OK
80                  // Receive "!ABC#" - temp_cmd = "ABC" -CMD_NONE
81                  if (strcmp((char *)temp_cmd, "RST") == 0)
82                  {
83                      command_flag = CMD_RST;
84                      strcpy((char *)command_data, "RST");
85                  }
86                  else if (strcmp((char *)temp_cmd, "OK") == 0)
87                  {
88                      command_flag = CMD_OK;
89                      strcpy((char *)command_data, "OK");
90                  }
91                  else
92                  {
93                      // Invalid command - Do nothing
94                      command_flag = CMD_NONE;
95                  }
96                  // Reset FSM to wait for new command
97                  parser_state = PARSER_INIT;
98                  cmd_index = 0;
99              }
100             // CASE 2: Received '!' in the middle
101             else if (received_char == '!')
102             {
103                 // Restart: Begin receiving new command
104                 cmd_index = 0;
```

```
                memset(temp_cmd, 0, MAX_CMD_LENGTH);
            }

            // CASE 3: Received normal character (R, S, T...)
            else
            {
                if (cmd_index < MAX_CMD_LENGTH - 1)
                {
                    temp_cmd[cmd_index++] = received_char;
                }
                else
                {
                    parser_state = PARSER_INIT;  // Buffer full
                    cmd_index = 0;
                }
            }
            break;

        // DEFAULT STATE: Error handling
        default:
            parser_state = PARSER_INIT; // Reset to safe state
            break;
        }
    }
}
// Get the parsed command
uint8_t get_command_flag(void)
{
    return command_flag;
}
// Clear command flag
void clear_command_flag(void)
{
    command_flag = CMD_NONE;
    memset(command_data, 0, MAX_CMD_LENGTH);
}
```

2. `uart_communication.c`

```c
/* uart_communication.c - UART communication protocol management using FSM
 * PROTOCOL: PC sends !RST# => STM32 sends !ADC=xxxx# => PC sends !OK# (3s timeout)
 * If timeout => Retransmit with previous value */

#include "uart_communication.h"
#include "command_parser.h"
#include "sensor.h"
#include "stdio.h"
#include "string.h"

// GLOBAL VARIABLES
static CommunicationState comm_state = COMM_IDLE;
static uint32_t timeout_start = 0 // Timeout start timestamp (ms)
static uint32_t last_adc_value = 0;// Last transmitted ADC value

// INITIALIZATION
void uart_communication_init(void)
{
    comm_state = COMM_WAIT_RST;  // Start by waiting for RST command
    timeout_start = 0;
    last_adc_value = 0;
}

// Send ADC value via UART in format !ADC=xxxx#
void send_adc_packet(uint32_t value)
{
    char packet[50];
    // Format packet: !ADC=xxxx#
    sprintf(packet, "!ADC=%lu#\r\n", value);
    // Transmit via UART2
    HAL_UART_Transmit(&huart2, (uint8_t *)packet,
                            strlen(packet), 1000);

```

```
34      // Store value for potential retransmission
35      last_adc_value = value;
36  }
37
38  /* Main FSM for managing UART communication
39   STATES:
40   - COMM_IDLE: Initial state, transitions to COMM_WAIT_RST
41   - COMM_WAIT_RST: Waiting for !RST# command from PC
42   - COMM_WAIT_OK: Waiting for !OK# confirmation (3 second timeout)
43   - COMM_TIMEOUT: Timeout occurred, retransmit previous value
44   */
45  void uart_communication_fsm(void)
46  {
47      // Get current command from parser
48      uint8_t cmd = get_command_flag();
49
50      switch (comm_state)
51      {
52      // STATE 1: COMM_IDLE - Initial idle state
53      case COMM_IDLE:
54          comm_state = COMM_WAIT_RST;  // Transition immediately
55          break;
56
57      // STATE 2: COMM_WAIT_RST - Waiting for !RST# from PC
58      case COMM_WAIT_RST:
59          if (cmd == CMD_RST)
60          {
61              // Clear the command flag
62              clear_command_flag();
63              // Read ADC value from sensor
64              adc_value = read_adc_value();
65
66          // Send debug info (optional)
67          char debug_msg[50];
68          sprintf(debug_msg, "[INFO] ADC Read: %lu\r\n", adc_value);
69          HAL_UART_Transmit(&huart2, (uint8_t *)debug_msg,
```

```
                    strlen(debug_msg), 100);

            // Send ADC packet to PC
            send_adc_packet(adc_value);
            // Start timeout counter
            timeout_start = HAL_GetTick();
            // Transition to waiting for OK
            comm_state = COMM_WAIT_OK;
        }
        break;

    // STATE 3: COMM_WAIT_OK - Waiting for !OK# confirmation
    case COMM_WAIT_OK:
        // Case 1: Received !OK# => Success
        if (cmd == CMD_OK)
        {
            clear_command_flag();
            // Send success message
            char success_msg[] = "[SUCCESS] Communication complete
                                                    !\r\n\r\n";
            HAL_UART_Transmit(&huart2, (uint8_t *)success_msg,
                                    strlen(success_msg), 100);
            // Return to waiting for next RST
            comm_state = COMM_WAIT_RST;
        }

        // Case 2: Timeout (3 seconds elapsed)
        else if (HAL_GetTick() - timeout_start >= TIMEOUT_DURATION)
        {
            comm_state = COMM_TIMEOUT;  // Handle timeout
        }
        break;
    // STATE 4: COMM_TIMEOUT - Handle timeout and retransmit
    case COMM_TIMEOUT:
    {
        // Send timeout notification
```

```
103         char timeout_msg[] = "[TIMEOUT] The value is kept as the
    previous packet.\r\n";
104         HAL_UART_Transmit(&huart2, (uint8_t *)timeout_msg, strlen(
    timeout_msg), 100);
105 
106         // Retransmit with previous value
107         send_adc_packet(last_adc_value);
108 
109         // Restart timeout counter
110         timeout_start = HAL_GetTick();
111 
112         // Return to waiting for OK
113         comm_state = COMM_WAIT_OK;
114         break;
115     }
116 
117     // DEFAULT: Error recovery
118     default:
119         comm_state = COMM_IDLE;  // Reset to safe state
120         break;
121     }
122 }
123 
124 // Get current FSM state
125 CommunicationState get_communication_state(void)
126 {
127     return comm_state;
128 }
```

3. `sensor.c`

```c
#include "sensor.h"
// CONSTANTS
#define ADC_MAX_VALUE 4095    // Maximum value for 12-bit ADC
#define VREF_MV 3300          // Reference voltage 3.3V = 3300mV

// INITIALIZATION
void sensor_init(void) {
    HAL_ADC_Start(&hadc1);
}

// READ ADC VALUE
uint32_t read_adc_value(void) {
    HAL_Delay(1);  // Short delay to ensure fresh value

    // Read and return ADC value
    return HAL_ADC_GetValue(&hadc1);
}

// CONVERT ADC TO VOLTAGE
uint32_t adc_to_voltage_mv(uint32_t adc_value) {
    uint32_t voltage_mv = (adc_value * VREF_MV) / ADC_MAX_VALUE;
    return voltage_mv;
}
```

Tài liệu

[1]